Theory of Programming Languages

3. Simply Typed Lambda Calculus

Kihong Heo

KAIST

Type

- Role of types in PL: framework for understanding and designing PLs
  - Syntax: constructors for each type
  - Semantics: coherence between static and dynamic semantics
- Harmony between two aspects of programs
  - Dynamic semantics (동적, 실행의미): runtime behavior (e.g., operational semantics)
  - Static semantics (정적, 실행전의미): estimation of runtime behavior (e.g., type system)

Simply Typed Lambda Calculus단순 타입 람다계산법

- Introduced by Alonzo Church (1940)
- Restricted and conservative: ill-typed terms excluded
- Syntax: type annotation for all lambda abstraction
  - Base type unit or function type

$$\begin{array}{llll} \text{Term} & E & \rightarrow & x \mid \lambda x : \tau . E \mid E\ E \mid \langle\rangle \\ \text{Type} & \tau & \rightarrow & \mathsf{unit} \mid \tau \rightarrow \tau \end{array}$$

Dynamic Semantics동적 의미구조

- Term, type, context, value

$$
\begin{array}{llll}
\text{Term} & E & \to & x \mid \lambda x{:}\tau.E \mid E\ E \mid \langle\rangle \\
\text{Type} & \tau & \to & \mathsf{unit} \mid \tau \to \tau \\
\text{Ctx} & K & \to & [\,] \mid K\ E \mid v\ E \\
\text{Value} & v & \to & \langle\rangle \mid \lambda x{:}\tau.E
\end{array}
$$

- Operational semantics

$$\frac{}{(\lambda x.E)\ v \to E\{v/x\}}$$

$$\frac{E \to E'}{K[E] \to K[E']}$$

Static Semantics정적 의미구조

- Proof of judgement $\Gamma \vdash E : \tau$ means "E has type τ in environment Γ"
- Type environment $\Gamma \in \text{Var} \to \text{Type}$
- Typing rules

$$\frac{}{\Gamma \vdash \langle\rangle : \text{unit}} \qquad \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \qquad \frac{\Gamma + x : \tau_1 \vdash E : \tau_2}{\Gamma \vdash \lambda x{:}\tau_1.E : \tau_1 \to \tau_2} \qquad \frac{\Gamma \vdash E_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash E_2 : \tau_1}{\Gamma \vdash E_1\ E_2 : \tau_2}$$

Dynamic $\approx$ Static

- Consistency between dynamic and static semantics?
  - Soundness (안전성): well-typed (static) $\Longrightarrow$ well-behaved (dynamic)
  - Completeness (완전성): well-behaved (dynamic) $\Longrightarrow$ well-typed (static)
- Impossible to achieve both (i.e., undecidable, 계산불가능성)
- Typically, sound type systems are used

Soundness

Theorem (Soundness). For all terms E, if $\vdash E : \tau$ and $E \to^* E'$, then $E' \in$ Value or $\exists E''\,.\,E' \to E''$

Lemma (Preservation). For all terms E, if $\vdash E : \tau$ and $E \to E'$, then $\vdash E' : \tau$

Lemma (Progress). For all terms E, if $\vdash E : \tau$ then $E \in$ Value or $\exists E'\,.\,E \to E'$

Enriching the Language

- Extend the language with values and their types
- Add new types = provide ways for construction and use for each type
  - Introduction/elimination (production/consumption)
- Example: function type
  - Introduction: $\lambda x : \tau \,.\, E$ (abstraction)
  - Elimination: $E\ E$ (application)

Let Binding

- Terms and evaluation contexts

$$\begin{array}{rcl} E & \to & \ldots \mid \texttt{let}\ x\ \texttt{=}\ E\ \texttt{in}\ E \\ K & \to & \ldots \mid \texttt{let}\ x\ \texttt{=}\ K\ \texttt{in}\ E \end{array}$$

- Dynamic semantics

$$\frac{E \to E'}{K[E] \to K[E']}$$

$$\frac{}{\texttt{let}\ x\ \texttt{=}\ v\ \texttt{in}\ E \to E\{v/x\}}$$

- Static semantics

$$\frac{\Gamma \vdash E_1 : \tau_1 \qquad \Gamma + x{:}\tau_1 \vdash E_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x\ \texttt{=}\ E_1\ \texttt{in}\ E_2 : \tau_2}$$

Product Type (1)

- Terms, evaluation contexts, values, and types

$$
\begin{array}{rcl}
E & \to & \ldots \mid \langle E, E \rangle \mid E.1 \mid E.2 \\
K & \to & \ldots \mid \langle K, E \rangle \mid \langle v, K \rangle \mid K.1 \mid K.2 \\
v & \to & \ldots \mid \langle v, v \rangle \\
\tau & \to & \ldots \mid \tau \times \tau
\end{array}
$$

- Dynamic semantics

$$
\frac{E \to E'}{K[E] \to K[E']}
$$

$$
\frac{}{\langle v_1, v_2 \rangle.1 \to v_1} \qquad \frac{}{\langle v_1, v_2 \rangle.2 \to v_2}
$$

Product Type (2)

- Static semantics

$$\frac{\Gamma \vdash E_1 : \tau_1 \qquad \Gamma \vdash E_2 : \tau_2}{\Gamma \vdash \langle E_1, E_2 \rangle : \tau_1 \times \tau_2}$$

$$\frac{\Gamma \vdash E : \tau_1 \times \tau_2}{\Gamma \vdash E.1 : \tau_1} \qquad \frac{\Gamma \vdash E : \tau_1 \times \tau_2}{\Gamma \vdash E.2 : \tau_2}$$

Tuple Type (1)

- Generalization of binary products (i.e., n-ary product)
- Terms, evaluation contexts, values, and types

$$
\begin{array}{rcl}
E & \to & \ldots \mid \langle E_i \ ^{i \in 1 \ldots n} \rangle \mid E.i \\
K & \to & \ldots \mid \langle v_i \ ^{i \in 1 \ldots j-1}, K, E_k \ ^{k \in j+1 \ldots n} \rangle \mid K.i \\
v & \to & \ldots \mid \langle v_i \ ^{i \in 1 \ldots n} \rangle \\
\tau & \to & \ldots \mid \langle \tau_i \ ^{i \in 1 \ldots n} \rangle
\end{array}
$$

- Dynamic semantics

$$\frac{E \to E'}{K[E] \to K[E']}$$

$$\frac{}{\langle v_i \ ^{i \in 1 \ldots n} \rangle.j \to v_j}$$

Tuple Type (2)

- Static semantics

$$\frac{\text{for each } i \qquad \Gamma \vdash E_i : \tau_i}{\Gamma \vdash \langle E_i \ ^{i \in 1 \ldots n} \rangle : \langle \tau_i \ ^{i \in 1 \ldots n} \rangle}$$

$$\frac{\Gamma \vdash E : \langle \tau_i \ ^{i \in 1 \ldots n} \rangle}{\Gamma \vdash E.j : \tau_j}$$

Record Type (1)

- Generalization of tuples (i.e., tuples with labels)
- Terms, evaluation contexts, values, and types

$$
\begin{array}{rcl}
E & \to & \ldots \mid \{\mathtt{l}_i = E_i \ ^{i \in 1 \ldots n}\} \mid E.\mathtt{l} \\
K & \to & \ldots \mid \{\mathtt{l}_1 = v_1 \ ^{i \in 1 \ldots j-1}, \mathtt{l}_j = K, \mathtt{l}_k = E_k \ ^{k \in j+1 \ldots n}\} \mid K.\mathtt{l} \\
v & \to & \ldots \mid \{\mathtt{l}_i = v_i \ ^{i \in 1 \ldots n}\} \\
\tau & \to & \ldots \mid \{\mathtt{l}_i : \tau_i \ ^{i \in 1 \ldots n}\}
\end{array}
$$

- Dynamic semantics

$$\frac{E \to E'}{K[E] \to K[E']}$$

$$\frac{}{\{\mathtt{l}_i = v_i \ ^{i \in 1 \ldots n}\}.\mathtt{l}_j \to v_j}$$

Record Type (2)

- Static semantics

$$\frac{\text{for each } i \qquad \Gamma \vdash E_i : \tau_i}{\Gamma \vdash \{\mathtt{l}_i = E_i \ ^{i \in 1 \ldots n}\} : \{\mathtt{l}_i : \tau_i \ ^{i \in 1 \ldots n}\}}$$

$$\frac{\Gamma \vdash E : \{\mathtt{l}_i : \tau_i \ ^{i \in 1 \ldots n}\}}{\Gamma \vdash E.\mathtt{l}_j : \tau_j}$$

Sum Type (1)

- Terms, evaluation contexts, values, and types

$$
\begin{array}{rcl}
E & \to & \ldots \mid \texttt{inl}\ E \mid \texttt{inr}\ E \\
 & \mid & \texttt{case}\ E\ \texttt{of}\ \texttt{inl}\ x \Rightarrow E\ \texttt{|}\ \texttt{inr}\ x \Rightarrow E \\
K & \to & \ldots \mid \texttt{inl}\ K \mid \texttt{inr}\ K \\
 & \mid & \texttt{case}\ K\ \texttt{of}\ \texttt{inl}\ x \Rightarrow E\ \texttt{|}\ \texttt{inr}\ x \Rightarrow E \\
v & \to & \ldots \mid \texttt{inl}\ v \mid \texttt{inr}\ v \\
\tau & \to & \ldots \mid \tau + \tau
\end{array}
$$

- Dynamic semantics

$$\frac{E \to E'}{K[E] \to K[E']}$$

$$\frac{}{\texttt{case}\ (\texttt{inl}\ v)\ \texttt{of}\ \texttt{inl}\ x \Rightarrow E_1\ \texttt{|}\ \texttt{inr}\ x \Rightarrow E_2 \to E_1\{v/x\}}$$

$$\frac{}{\texttt{case}\ (\texttt{inr}\ v)\ \texttt{of}\ \texttt{inl}\ x \Rightarrow E_1\ \texttt{|}\ \texttt{inr}\ x \Rightarrow E_2 \to E_2\{v/x\}}$$

Sum Type (2)

- Static semantics

$$\frac{\Gamma \vdash E : \tau_1}{\Gamma \vdash \texttt{inl}\ E : \tau_1 + \tau_2} \qquad \frac{\Gamma \vdash E : \tau_2}{\Gamma \vdash \texttt{inr}\ E : \tau_1 + \tau_2}$$

$$\frac{\Gamma \vdash E : \tau_1 + \tau_2 \qquad \Gamma + x : \tau_1 \vdash E_1 : \tau \qquad \Gamma + x : \tau_2 \vdash E_2 : \tau}{\Gamma \vdash \texttt{case}\ E\ \texttt{of inl}\ x\ \Rightarrow E_1\ \texttt{|}\ \texttt{inr}\ x \Rightarrow E_2 : \tau}$$

- Problem?
- Example: What is the type of $\texttt{inl}\ 5$?

Sum Type (3)

- Explicit type annotation to sum type

$$
\begin{array}{rcl}
E & \to & \ldots \mid \texttt{inl}\ E\ \texttt{as}\ \tau \mid \texttt{inr}\ E\ \texttt{as}\ \tau \\
 & \mid & \texttt{case}\ E\ \texttt{of}\ \texttt{inl}\ x \Rightarrow E\ \texttt{|}\ \texttt{inr}\ x \Rightarrow E \\
K & \to & \ldots \mid \texttt{inl}\ K\ \texttt{as}\ \tau \mid \texttt{inr}\ K\ \texttt{as}\ \tau \\
 & \mid & \texttt{case}\ K\ \texttt{of}\ \texttt{inl}\ x \Rightarrow E\ \texttt{|}\ \texttt{inr}\ x \Rightarrow E \\
v & \to & \ldots \mid \texttt{inl}\ v\ \texttt{as}\ \tau \mid \texttt{inr}\ v\ \texttt{as}\ \tau \\
\tau & \to & \ldots \mid \tau + \tau
\end{array}
$$

- Static semantics

$$
\frac{\Gamma \vdash E : \tau_1}{\Gamma \vdash \texttt{inl}\ E\ \texttt{as}\ \tau_1 + \tau_2 : \tau_1 + \tau_2} \qquad \frac{\Gamma \vdash E : \tau_2}{\Gamma \vdash \texttt{inr}\ E\ \texttt{as}\ \tau_1 + \tau_2 : \tau_1 + \tau_2}
$$

$$
\frac{\Gamma \vdash E : \tau_1 + \tau_2 \qquad \Gamma + x : \tau_1 \vdash E_1 : \tau \qquad \Gamma + x : \tau_2 \vdash E_2 : \tau}{\Gamma \vdash \texttt{case}\ E\ \texttt{of}\ \texttt{inl}\ x\ \Rightarrow E_1\ \texttt{|}\ \texttt{inr}\ x \Rightarrow E_2 : \tau}
$$

Variant Type (1)

- Generalization of binary sums
- Terms, evaluation contexts, values, and types

$$
\begin{array}{rcl}
E & \to & \ldots \mid \texttt{l}\ E\ \texttt{as}\ \tau \mid \texttt{case}\ E\ \texttt{of}\ \overline{\texttt{l}_i\ x_i \Rightarrow E_i}^{\,i \in 1 \ldots n} \\
K & \to & \ldots \mid \texttt{l}\ K\ \texttt{as}\ \tau \mid \texttt{case}\ K\ \texttt{of}\ \overline{\texttt{l}_i\ x_i \Rightarrow E_i}^{\,i \in 1 \ldots n} \\
v & \to & \ldots \mid \texttt{l}\ v\ \texttt{as}\ \tau \\
\tau & \to & \ldots \mid \texttt{<}\overline{\texttt{l}_i : \tau_i}^{\,i \in 1 \ldots n}\texttt{>}
\end{array}
$$

- Dynamic semantics

$$
\frac{E \to E'}{K[E] \to K[E']}
$$

$$
\frac{}{\texttt{case}\ \texttt{l}_j\ v_j\ \texttt{of}\ \overline{\texttt{l}_i\ x_i \Rightarrow E_i}^{\,i \in 1 \ldots n} \to E_j\{v_j/x_j\}}
$$

Variant Type (2)

- Static semantics

$$\frac{\Gamma \vdash E_j : \tau_j}{\Gamma \vdash \texttt{l}_j\ E_j\ \texttt{as}\ \texttt{<l}_i : \tau_i\ ^{i \in 1 \ldots n}\texttt{>} : \texttt{<l}_i : \tau_i\ ^{i \in 1 \ldots n}\texttt{>}}$$

$$\frac{\Gamma \vdash E : \texttt{<l}_i : \tau_i\ ^{i \in 1 \ldots n}\texttt{>} \qquad \text{for each } i \qquad \Gamma + x_i : \tau_i \vdash E_i : \tau}{\Gamma \vdash \texttt{case}\ E\ \texttt{of}\ \texttt{l}_i\ x_i \Rightarrow E_i\ ^{i \in 1 \ldots n} : \tau}$$

Recursion

- Type of $\Omega = (\lambda x.x\ x)(\lambda x.x\ x)$
- Type of $\texttt{fix} = \lambda f.(\lambda x.f\ (\lambda y.x\ x\ y))\ (\lambda x.f\ (\lambda y.x\ x\ y))$
- Type of $(x\ x)$

Inhabitation

- If there exists a term E such that $\varnothing \vdash E : \tau$ then E is an inhabitant of τ
  - unit : $\{\langle\rangle\}$
  - bool : $\{\texttt{true}, \texttt{false}\}$
  - nat : $\{0,1,2,\ldots\}$
  - nat $\rightarrow$ nat : $\{\texttt{succ}, \ldots\}$
- Uninhabited type?
  - $\tau : \varnothing$?

Void Type (1)

- Uninhabited type (i.e., no values of void type exist)
  - Empty type
  - Dual of unit type: unit has only one value, whereas void has none
- No values and dynamic semantics

$$E \rightarrow \ldots \mid \texttt{abort}_\tau\ E \qquad \texttt{abort}_\tau\ E = \texttt{case}\ E\ \texttt{of}\ \mid$$
$$\tau \rightarrow \ldots \mid \texttt{void}$$

- Static semantics (only elimination)

$$\frac{\Gamma \vdash E : \texttt{void}}{\Gamma \vdash \texttt{abort}_\tau\ E : \tau}$$

Void Type (2)

- Example: $\lambda x : \texttt{void} + \texttt{unit}.\ \texttt{case}\ x\ \texttt{of}\ \texttt{inl}\ y \Rightarrow \texttt{abort}(y)\ \mid\ \texttt{inr}\ y \Rightarrow y$
- “Nothing” in Scala, “never” in Rust
- Simulation in OCaml

```
type void = |
type ('a, 'e) t = Ok of 'a | Error of 'e
type ('a, 'e) operation = int -> int -> ('a, 'e) t

let add : (int, void) operation = fun x y -> Ok (x + y)
let sub : (int, void) operation = fun x y -> Ok (x - y)
let mul : (int, void) operation = fun x y -> Ok (x * y)
let div : (int, string) operation =
 fun x y -> if y = 0 then Error "Division by zero" else Ok (x / y)

let log_result name op x y =
  match op x y with
  | Ok v -> Printf.printf "%s(%d, %d) = %d\n" name x y v; Ok v
  | Error e -> Printf.printf "%s(%d, %d) failed\n" name x y; Error e

let _ = log_result "add" add 10 5
let _ = log_result "div" div 10 0
```

```
let run_infallible (op : ('a, void) operation) x y =
  match op x y with Ok v -> v | Error impossible -> .

let run_fallible op x y =
  match op x y with
  | Ok v -> v
  | Error e -> prerr_endline e; exit 1

let _ = run_infallible add 10 5
let _ = run_fallible div 10 0
```

Normalization

- Every well-typed term is normalizable in STLC
  - The evaluation of a well-typed program eventually halts
- Not true in general
- Proof! (Excecise)

Summary

- Simple types: unit, void, function, product, sum, etc
- Conservative but restricted: reject many “meaningful” programs
- Normalization: the evaluation of a well-typed program eventually halts